Table des matières

Labo 03 — Monter le projet dbt et le STAGING 1
Avant de commencer 2
Étape 1 — Le fichier de connexion, hors du dépôt 2
Étape 2 — Le projet dbt, à la racine du dépôt 3
Étape 3 — La macro des schémas 4
Étape 4 — Déclarer les sources 4
Étape 5 — Voir ce que dbt connaît 5
Étape 6 — Les premières vues de staging 5
Étape 7 — La vue qui retrouve les 25 483 commandes 7
Étape 8 — Relier deux modèles, et voir le graphe 9
Dépannage 10

Labo 03 — Monter le projet dbt et le STAGING

Labo · Cours 3/9 — Formation Analytics Engineering

But : monter le projet dbt dans le dépôt `02_ae`, brancher sa connexion hors du dépôt, déclarer la `RAW` comme source, puis écrire les premières vues du `STAGING`, dont celle qui retrouve les 25 483 commandes de Boréal malgré l'extrait livré deux fois, et un contrôle qui relie deux vues. Le pourquoi de chaque étape est dans `Theorie_03_AE.md`, annexe A.

Deux fenêtres de l'Anaconda Prompt, toutes deux placées dans `Projet_Boreal_intermediaire`, environnement `formations` actif :

- la première pour le Bloc-notes, dbt et Git ;
- la seconde pour le notebook de DuckDB (`duckdb -ui`), « AE Labo 03 », qui lit ce que dbt a bâti. Elle s'ouvre à l'étape 6.

Un fichier DuckDB ouvert par un processus bloque l'écriture des autres, même en lecture seule : le notebook détache l'entrepôt avant chaque commande dbt, et le rattache ensuite.

Liste de contrôle :

- ☐ `profiles.yml` dans `Projet_Boreal_intermediaire`, hors du dépôt

© 2026 iAlexMG. Tous droits réservés.

- ☐ `dbt_project.yml` à la racine de `02_ae`, et `dbt debug` au vert
- ☐ La macro `generate_schema_name` en place : les vues dans `staging`, pas `staging_staging`
- ☐ Deux sources déclarées, et `dbt ls` qui les montre
- ☐ `stg_concessionnaires` : 156 lignes, 0 palier vide, 4 fiches fermées
- ☐ À vous : `stg_articles`, 68 articles, 0 gamme vide
- ☐ `stg_commandes` : 25 483 lignes, malgré les 26 538 de la source
- ☐ Le contrôle à 0, le graphe relié, le travail validé dans `02_ae`

Avant de commencer

La couche `RAW` vient du Cours 00, `Cours_00_AE_Boreal.md`, et le dépôt `02_ae` du Cours 01. Dans la première fenêtre :

```
set DBT_PROJECT_DIR=02_ae
REM Attendu : rien ne s'affiche.
dir /b
REM Attendu : 01_de, 02_ae, plateforme et sources, au moins.
dbt --version
REM Attendu : Core installed: 1.12.3 ou plus récent, et duckdb: 1.11.0 sous Plugins.
```

`set DBT_PROJECT_DIR=02_ae` dit à dbt de chercher son projet dans `02_ae` : la fenêtre reste dans `Projet_Boreal_intermediaire`, à côté de l'entrepôt. La variable ne vit que dans cette fenêtre, et chaque cours la redonne.

Le nom n'est pas inventé : `DBT_PROJECT_DIR` est la variable que dbt lit à la place de son option `--project-dir`, et sa documentation la donne sous ce nom. D'autres options de dbt ont ainsi leur variable, `DBT_` suivi du nom de l'option : `DBT_PROFILES_DIR` remplace `--profiles-dir`. Sans la variable, chaque commande devrait finir par `--project-dir 02_ae`.

Étape 1 — Le fichier de connexion, hors du dépôt

Théorie : `Theorie_03_AE.md`, annexe A.1.

1.1 Écrire la connexion, dans `Projet_Boreal_intermediaire`, hors du dépôt.

```
notepad profiles.yml
```


© 2026 iAlexMG. Tous droits réservés.

Le Bloc-notes propose de créer le fichier : accepter. Écrire, enregistrer, fermer :

```
boreal_ae:
  target: dev
  outputs:
    dev:
      type: duckdb
      path: plateforme/entrepot/boreal.duckdb
      schema: staging
      threads: 4
```

Étape 2 — Le projet dbt, à la racine du dépôt

Théorie : `Theorie_03_AE.md`, annexe A.2.

2.1 Écrire le fichier du projet.

```
notepad 02_ae\dbt_project.yml
```

Accepter la création, puis écrire, enregistrer, fermer :

```
name: boreal_ae
version: 1.0.0
profile: boreal_ae

model-paths: ["models"]
macro-paths: ["macros"]
test-paths: ["tests"]
target-path: "target"
clean-targets: ["target", "dbt_packages"]

models:
  boreal_ae:
    staging:
      +materialized: view
      +schema: staging
```

2.2 Créer les dossiers des modèles et des macros.

```
mkdir 02_ae\models\staging 02_ae\macros
```

2.3 Vérifier le projet et la connexion.

© 2026 iAlexMG. Tous droits réservés.

```
dbt debug
REM Attendu : profiles.yml file [OK found and valid], dbt_project.yml file [OK found and valid],
REM Connection test: [OK connection ok], puis All checks passed!
```

Étape 3 — La macro des schémas

Théorie : Theorie_03_AE.md, annexe A.3.

3.1 Écrire la macro qui décide où dbt écrit.

```
notepad 02_ae\macros\generate_schema_name.sql
```

Accepter la création, puis écrire, enregistrer, fermer :

```
{% macro generate_schema_name(custom_schema_name, node) -%}

    {%- if custom_schema_name is none -%}
        {{ target.schema }}
    {%- else -%}
        {{ custom_schema_name | trim }}
    {%- endif -%}

{%- endmacro %}
```

Étape 4 — Déclarer les sources

Théorie : Theorie_03_AE.md, annexe A.4.

4.1 Déclarer les deux tables que les vues de ce labo liront d'abord.

```
notepad 02_ae\models\staging\_sources.yml
```

Accepter la création, puis écrire, enregistrer, fermer :

```
version: 2

sources:
  - name: raw
    description: "La couche RAW livrée par Data Engineering, 22 tables."
    schema: raw
    tables:
      - name: erp_kna1
        description: "Le référentiel des concessionnaires, un extrait par mois."
```


© 2026 iAlexMG. Tous droits réservés.

```
      - name: erp_vbak
        description: "Les en-têtes de commande."
```

Étape 5 — Voir ce que dbt connaît

Théorie : `Theorie_03_AE.md`, annexe A.5.

5.1 Lister ce que dbt connaît du projet.

```
dbt ls
REM Attendu : source:boreal_ae.raw.erp_kna1 et source:boreal_ae.raw.erp_vbak, précédées d'un
REM avertissement, Configuration paths exist in your dbt_project.yml file which do not apply to
REM any resources.
```

Étape 6 — Les premières vues de staging

Théorie : `Theorie_03_AE.md`, annexe A.6.

6.1 Écrire la vue des concessionnaires.

```
notepad 02_ae\models\staging\stg_concessionnaires.sql
```

Accepter la création, puis écrire, enregistrer, fermer :

```
with courant as (

    select
        *,
        row_number() over (
            partition by kunnr
            order by _lot desc
        ) as rang

    from {{ source('raw', 'erp_kna1') }}

)

select
    kunnr as code_concessionnaire,
    name1 as nom_concessionnaire,
    ort01 as ville,
    regio as province,
    case kukla
```


© 2026 iAlexMG. Tous droits réservés.

```
            when '01' then 'Or'
            when '02' then 'Argent'
            when '03' then 'Bronze'
        end as palier,
        try_strptime(erdat, '%Y%m%d')::date as date_creation,
        loevm = 'X' as est_supprime,
        _lot as lot_source

    from courant
    where rang = 1
```

6.2 À vous : écrire `stg_articles`, la vue des articles, par les mêmes gestes, et déclarer sa source, `erp_mara`, dans `_sources.yml`, avec une description. C'est le geste des vues du Cours 03 de La Brûlerie, sans son code. Six colonnes :

Colonne	D'où elle vient
`code_article`	`matnr`, la clé de l'état courant
`gamme`	`matkl` décodé : quatre codes, quatre gammes
`type_article`	`mtart` décodé : deux codes, deux types
`annee_modele`	`zzannee_mod`, vide pour une pièce
`unite_de_base`	`meins`
`lot_source`	`_lot`

Les colonnes se lisent à deux endroits, et chacun dit sa part :

Où regarder	Ce qu'on y trouve	Ce qu'on n'y trouve pas
`Data_Domain/_univers/boreal/DICTIONNAIRE.md`, section de `MARA`, puis celles de `MATKL` et de `MTART` pour les codes	chaque champ, son type SAP et son sens ; le sens de chaque code	`_fichier`, `_lot` et `_charge_le`, que le chargeur de Data Engineering ajoute
le notebook, une fois ouvert (6.4) : `describe boreal.raw.erp_mara`	les colonnes telles que la `RAW` les garde, sous leur nom SAP, les trois de lignage en dernier	leur vrai type : la `RAW` garde tout en `VARCHAR`, du texte, sauf l'heure du chargement

DuckDB ne distingue pas les majuscules : `matnr` lit la colonne `MATNR`. Aucune des six colonnes ne se convertit : les deux codes se décodent par un `case`, et les autres, déjà du texte dans la `RAW`, montent telles quelles, sous leur nouveau nom. Le corrigé est `02_Analytics_Engineering/02_Boreal_intermediaire/Solution/02_ae/models/staging/stg_articles.sql`.

6.3 Bâtir les deux vues, dans la première fenêtre.

© 2026 iAlexMG. Tous droits réservés.

```
dbt run
REM Attendu : OK created sql view model staging.stg_articles, puis staging.stg_concessionnaires,
REM et Done. PASS=2 WARN=0 ERROR=0.
```

6.4 Ouvrir le notebook. Seconde fenêtre : activer `formations`, se placer dans `Projet_Boreal_intermediaire`, puis lancer DuckDB.

```
duckdb -ui
```

Attendu : l'interface s'ouvre dans le navigateur. Laisser cette fenêtre ouverte. Créer un notebook, le renommer « AE Labo 03 », puis attacher l'entrepôt dans la première cellule :

```
-- Attendu : boreal paraît sous Attached databases, avec les schémas raw et staging.
attach if not exists 'plateforme/entrepot/boreal.duckdb' as boreal (read_only);
```

6.5 Compter les concessionnaires, et ce que le décodage a laissé vide.

```
-- Attendu : 156 concessionnaires, 0 palier vide, 4 fiches fermées.
select
    count(*) as concessionnaires,
    count(*) - count(palier) as paliers_vides,
    count(case when est_supprime then 1 end) as fiches_fermees
from boreal.staging.stg_concessionnaires;
```

6.6 Compter les articles de votre vue.

```
-- Attendu : 68 articles, 0 gamme vide, 23 sans année-modèle.
select
    count(*) as articles,
    count(*) - count(gamme) as gammes_vides,
    count(*) - count(annee_modele) as sans_annee_modele
from boreal.staging.stg_articles;
```

6.7 Détacher l'entrepôt, pour que dbt puisse y écrire.

```
-- Attendu : boreal disparaît d'Attached databases.
detach database if exists boreal;
```

Étape 7 — La vue qui retrouve les 25 483 commandes

Théorie : `Theorie_03_AE.md`, annexe A.7.

7.1 Écrire la vue des commandes, dans la première fenêtre.

```
notepad 02_ae\models\staging\stg_commandes.sql
```


© 2026 iAlexMG. Tous droits réservés.

Accepter la création, puis écrire, enregistrer, fermer :

```
with courant as (

    select
        *,
        row_number() over (
            partition by vbeln
            order by _lot desc
        ) as rang

    from {{ source('raw', 'erp_vbak') }}

)

select
    vbeln as numero_commande,
    kunnr as code_concessionnaire,
    auart as type_document,
    try_strptime(erdat, '%Y%m%d')::date as date_creation,
    try_strptime(nullif(aedat, '00000000'), '%Y%m%d')::date as date_modification,
    netwr::decimal(18, 2) as valeur_nette,
    waerk as devise,
    _lot as lot_source

from courant
where rang = 1
```

7.2 Bâtir la vue.

```
dbt run
REM Attendu : Done. PASS=3 WARN=0 ERROR=0.
```

7.3 Rattacher l'entrepôt : relancer la cellule de 6.4.

7.4 Compter la `RAW` contre le `STAGING`, dans une cellule neuve. Une sous-requête entre parenthèses rend une seule valeur : elle s'apprend au Cours 07 de la formation SQL avancé, `Theorie_07_SQL.md`, section 2.

```
-- Attendu : 26 538 lignes dans la RAW, 25 483 commandes dans le STAGING, 2 231 modifiées.
select
    (select count(*) from boreal.raw.erp_vbak) as lignes_dans_la_raw,
    (select count(*) from boreal.staging.stg_commandes) as commandes_dans_le_staging,
    (select count(date_modification) from boreal.staging.stg_commandes) as commandes_modifiees;
```


© 2026 iAlexMG. Tous droits réservés.

7.5 Détacher l'entrepôt : relancer la cellule de 6.7.

Étape 8 — Relier deux modèles, et voir le graphe

Théorie : `Theorie_03_AE.md`, annexe A.8.

8.1 Écrire le contrôle, qui relie deux vues par `ref()`.

```
notepad 02_ae\models\staging\ctl_commandes_sans_concessionnaire.sql
```

Accepter la création, puis écrire, enregistrer, fermer :

```
select
    c.numero_commande,
    c.code_concessionnaire,
    c.date_creation

from {{ ref('stg_commandes') }} as c
left join {{ ref('stg_concessionnaires') }} as d
       on d.code_concessionnaire = c.code_concessionnaire

where d.code_concessionnaire is null
```

8.2 Bâtir le contrôle.

```
dbt run
REM Attendu : le contrôle se bâtit en dernier, après les deux vues qu'il lit ; Done. PASS=4.
```

8.3 Lister ce que dbt connaît maintenant.

```
dbt ls
REM Attendu : quatre modèles, boreal_ae.staging.ctl_commandes_sans_concessionnaire en tête, et
REM trois sources.
```

8.4 Rattacher l'entrepôt (cellule de 6.4), puis compter les commandes orphelines.

```
-- Attendu : 0.
select count(*) as commandes_sans_concessionnaire
from boreal.staging.ctl_commandes_sans_concessionnaire;
```

8.5 Voir l'orpheline que l'état courant a réparée.

```
-- Attendu : 0 fiche au lot 2024-05, première fiche au lot 2024-06, commande livrée au lot 2024-05.
select
```


© 2026 iAlexMG. Tous droits réservés.

```
(select count(*) from boreal.raw.erp_kna1 where kunnr = '0000200154' and _lot = '2024-05') as fiche_en_mai,
(select min(_lot) from boreal.raw.erp_kna1 where kunnr = '0000200154') as premiere_fiche,
(select _lot from boreal.raw.erp_vbak where vbeln = '0010015405') as lot_de_la_commande;
```

8.6 Fermer le notebook : fermer l'onglet du navigateur, puis `.quit` dans la seconde fenêtre.

```
.quit
```

8.7 Valider le travail, dans la première fenêtre.

```
git -C 02_ae add dbt_project.yml macros models
git -C 02_ae commit -m "Cours 03 : le projet dbt et la couche staging"
git -C 02_ae status
REM Attendu : nothing to commit, working tree clean.
```

Dépannage

Symptôme	Correctif
`dbt_project.yml file [ERROR not found]`	`set DBT_PROJECT_DIR=02_ae` manque dans cette fenêtre : le redonner
`profiles.yml file [ERROR not found]`, et le chemin d'un dossier `.dbt`	dbt n'est pas lancé depuis `Projet_Boreal_intermediaire` : y revenir, sans `cd` dans `02_ae`
`profiles.yml file [ERROR invalid]`, puis `Connection test skipped since no profile was found`	les deux noms diffèrent : comparer la première ligne de `profiles.yml` et la clé `profile` du projet
`cannot access local variable 'summary_message'`	`profiles.yml` existe mais est vide : le remplir
`IO Error: Cannot open file … utilisé par un autre processus`	le notebook tient l'entrepôt : relancer la cellule de 6.7, puis la commande dbt
`Binder Error: Catalog "boreal" does not exist!`	l'entrepôt est détaché : relancer la cellule de 6.4
Les vues atterrissent dans `staging_staging`	la macro de l'étape 3 manque, ou n'est pas dans `02_ae\macros`
`depends on a source named 'raw.erp_mara' which was not found`	`erp_mara` n'est pas déclarée dans `_sources.yml` (6.2)
`gammes_vides` n'est pas à 0	les codes du `case` ne sont pas ceux du dictionnaire

© 2026 iAlexMG. Tous droits réservés.